Demanda, Provisionamento e Desempenho

Grupo:
- João Victor Gouveia do Nascimento - 122210148
- José Ramon Severo Alves - 122210093

Aplicações desenvolvidas

Foram desenvolvidas três APIs de backend, cada uma com um recurso dominante diferente. A aplicação CPU Bound foi a escolhida para os experimentos na máquina virtual.

- **CPU Bound: Contador de números primos** (Python, FastAPI). `GET /primes/count?limit=N` conta os primos até N com o crivo de Eratóstenes. O custo de CPU cresce com N.
- **Memory Bound: API de escrita em memória** (NestJS, Bun). `GET /memory?valor=V` aloca um `Uint8Array` de 2^V bytes e o preenche. V deve ser um inteiro positivo, e qualquer valor é aceito nos testes de carga; o padrão é V = 24 (16 MiB). Porta 3000.
- **IO Bound: API de escrita em disco** (NestJS, Bun). `POST /write?valor=V` grava 2^V bytes em um arquivo, em blocos de 64 KiB, com `fsync` após cada bloco. O padrão é V = 20 (1 MiB, 16 sincronizações). Mesma pilha da aplicação anterior (NestJS em Bun); a porta é definida pela variável PORT (3001 nos nossos testes).

Configuração da VM e deploy da aplicação

Configuração da máquina virtual

A VM foi criada no VirtualBox a partir do disco virtual pré-configurado do Ubuntu Server 22.04 (VDI). As etapas foram:

1. Copiar o disco VDI para uma pasta local e criar uma nova VM do tipo Linux (Ubuntu 64-bit), associando o VDI como disco existente.
2. Definir os recursos iniciais da VM em 1 CPU e 1 GB de RAM. Esses valores mudam em cada cenário de teste.
3. Manter a rede em modo NAT e criar regras de redirecionamento de portas (a VM não é visível diretamente pela máquina hospedeira): porta 2222 do hospedeiro para a porta 22 da VM (SSH) e porta 8080 do hospedeiro para a porta 8000 da VM (aplicação).
4. Iniciar a VM e acessar com o usuário `osboxes`.
5. Instalar o servidor SSH e as ferramentas de monitoramento: `sudo apt install -y openssh-server sysstat htop`.

6. Validar o acesso externo: instalar o `nginx` na VM, acessar `localhost:80` de dentro dela e depois `localhost:8081` no hospedeiro, após criar a regra de redirecionamento correspondente.

Descrição da aplicação

A aplicação selecionada é o contador de números primos, uma API em FastAPI executada pelo servidor uvicorn. Ela expõe um único endpoint:

Item	Valor
Endpoint	`GET /primes/count?limit=N`
Parâmetro `limit`	inteiro entre 0 e 500.000.000 (padrão 1.000.000)
Resposta	`{"limit":1000000,"count":78498}`
Algoritmo	crivo de Eratóstenes em Python puro
Porta	8000

Características que influenciam o desempenho:

- **Uso intenso de CPU.** O crivo executa milhões de operações por requisição, e o custo cresce com `limit`. Praticamente não há espera por I/O.
- **Uso de memória proporcional a `limit`.** Cada requisição aloca uma lista de `limit + 1` elementos, cerca de 8 bytes cada (aprox. 4 MB para `limit = 500.000`). A memória cresce com o número de requisições simultâneas.
- **Limite de paralelismo por processo.** A função do endpoint é síncrona e roda no pool de threads do uvicorn. Pelo GIL do Python, um processo executa código Python em apenas um núcleo por vez. Para usar mais CPUs é preciso iniciar mais processos (`--workers`).

O projeto inclui um `Dockerfile` baseado em `python:3.14-slim`, que instala `fastapi` e `uvicorn[standard]` e inicia o servidor na porta 8000.

Deploy da aplicação

Requisitos na VM: Docker (pacote `docker.io`), acesso à internet para baixar a imagem base e as dependências, e a porta 8000 redirecionada para o hospedeiro. As etapas foram:

1. Instalar o Docker: `sudo apt install -y docker.io`, e adicionar o usuário ao grupo `docker`.
2. Copiar o projeto para a VM por SSH: `scp -P 2222 -r api-cpu-bound osboxes@localhost:~/` (sem a pasta `venv`).

3. Construir a imagem: `cd ~/api-cpu-bound && docker build -t cpu-bound-api .`
4. Iniciar o contêiner com o número de workers do cenário, usando o script auxiliar `start_app.sh`, que executa `docker run -d --rm -p 8000:8000 --name cpu-bound-api cpu-bound-api uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers N`.
5. Verificar a partir do hospedeiro: `curl "http://localhost:8080/primes/count?limit=1000000"` deve retornar `{"limit":1000000,"count":78498}`.

Os limites de CPU e memória não foram definidos no Docker. O contêiner usa os recursos da própria VM, que são alterados em cada cenário.

[Inserir capturas de tela: `docker build`, contêiner em execução e resposta do `curl` no hospedeiro.]

Teste de stress da aplicação selecionada

Cenários de configuração de recursos

Foram usados cinco cenários de provisionamento da VM. O número de workers do uvicorn acompanha o número de CPUs, exceto no cenário E, que serve para isolar o efeito do GIL.

Cenário	CPUs	RAM (GB)	Workers	Objetivo
A	1	1	1	linha de base
B	2	1	2	efeito de mais CPU
C	2	2	2	efeito de mais RAM (comparar com B)
D	4	2	4	efeito de mais CPU e workers
E	4	2	1	CPUs extras sem workers extras (limite do GIL)

Como foram realizados os testes de stress

A carga foi gerada fora da máquina virtual, no sistema hospedeiro, com o ApacheBench (`ab`) apontando para `localhost:8080`, porta redirecionada para a aplicação na VM. Assim, o gerador de carga não consome os recursos que estão sendo avaliados.

Procedimento (repetido em cada cenário)

1. Desligar a VM, ajustar CPUs e RAM com `VBoxManage modifyvm`, ligar a VM e iniciar o contêiner com o número de workers do cenário.

2. Fazer um aquecimento de 10 s com 10 clientes simultâneos, sem registrar os resultados.
3. Para cada nível de demanda (1, 5, 10, 25, 50, 100 e 200 clientes simultâneos), executar o teste por 30 s, com 3 repetições e pausa de 5 s entre execuções (cenários A, B e C). Nos cenários D e E, por falta de tempo, foram usadas 1 repetição de 20 s por nível, aquecimento de 5 s e pausa de 3 s.
4. Usar sempre a mesma requisição: `GET /primes/count?limit=750000`. O valor de `limit` foi escolhido de forma que uma requisição isolada leve de 30 a 100 ms, e foi mantido em todos os cenários.
5. Registrar as métricas e calcular a média das três repetições de cada nível.

Métricas coletadas

Métrica	Onde foi medida	Ferramenta
Vazão (requisições/s)	hospedeiro	`ab`
Latência média e percentil 99 (ms)	hospedeiro	`ab`
Requisições com falha	hospedeiro	`ab`
Uso de CPU da VM (média e pico, %)	dentro da VM	`vmstat 1`
Memória usada (MB) e atividade de swap	dentro da VM	`vmstat 1`

As ferramentas de monitoramento seguem a lista de ferramentas de saúde do Linux do MakeUseOf: `vmstat` (CPU, memória e swap em uma única saída), `htop` (inspeção visual por processo) e `sar`/`mpstat` do pacote `sysstat` (CPU ao longo do tempo e por núcleo). A coleta usa `vmstat` a cada segundo durante toda a execução.

Como o `ab` mantém um número fixo de clientes simultâneos, o número de requisições no sistema é igual à concorrência. Pela Lei de Little, a latência média é aproximadamente a concorrência dividida pela vazão. Assim, quando a vazão deixa de crescer (saturação), a latência passa a crescer de forma linear com o número de clientes. Essa relação, da teoria das filas (tema da palestra System Performance and Queuing Theory, SREcon24 Americas), orienta a interpretação dos resultados.

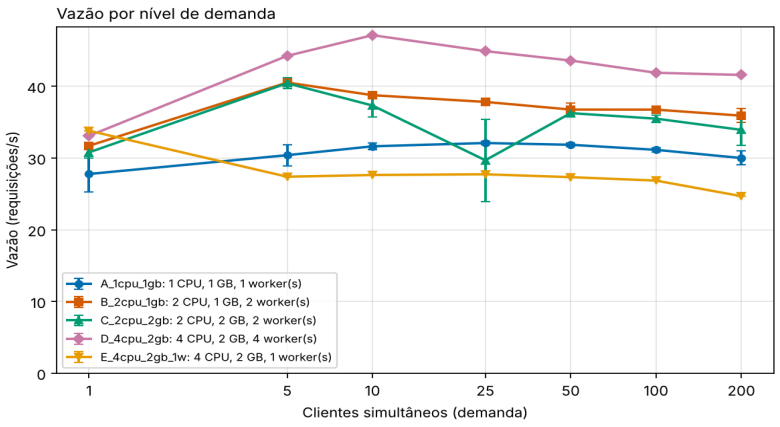
Resultados comparativos entre os cenários

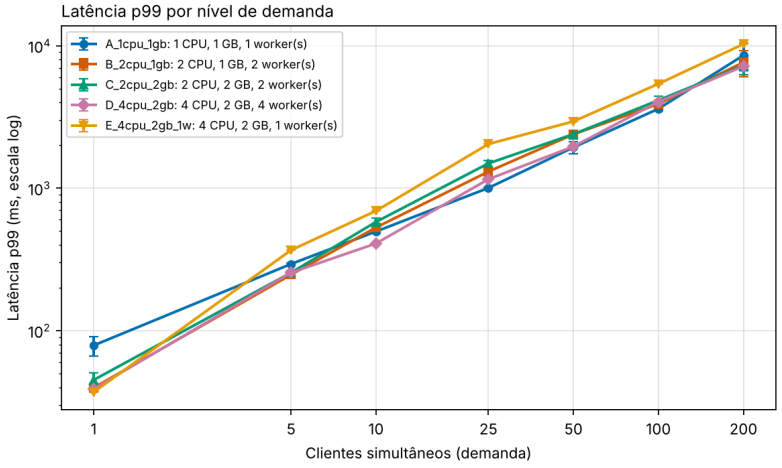
Status: completo, com ressalva de protocolo. Os cinco cenários foram executados. A, B e C seguem o protocolo completo (3 repetições de 30 s por nível de demanda); D e E, por falta de tempo, usaram um protocolo reduzido (1 repetição de 20 s por nível, aquecimento de 5 s), e por isso seus números são menos confiáveis. Os valores são médias do trecho estável de cada execução (sem aquecimento), com `limit` = 750000 (calibrado em ≈ 44 ms por requisição na VM de 1 CPU; sob carga, a vazão medida equivale a ≈ 31 ms). Nenhuma das 77 execuções teve requisições com falha. O swap na VM ocorreu apenas no cenário B (média de 3,5 MB com 200 clientes); nos demais foi zero.

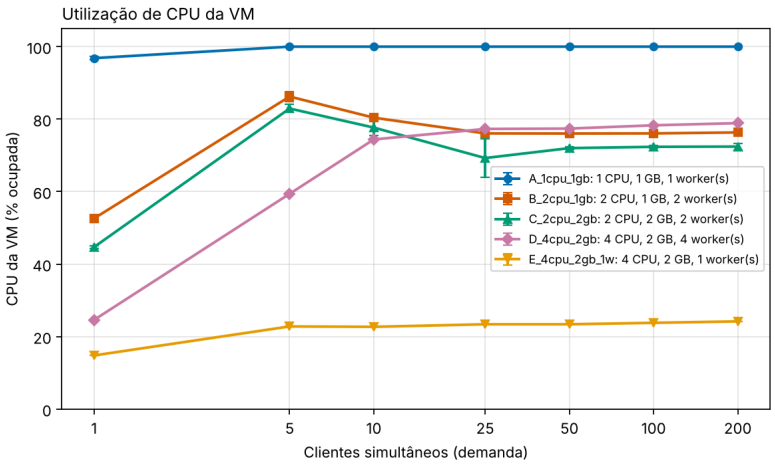
Cenário A: 1 CPU, 1 GB, 1 worker

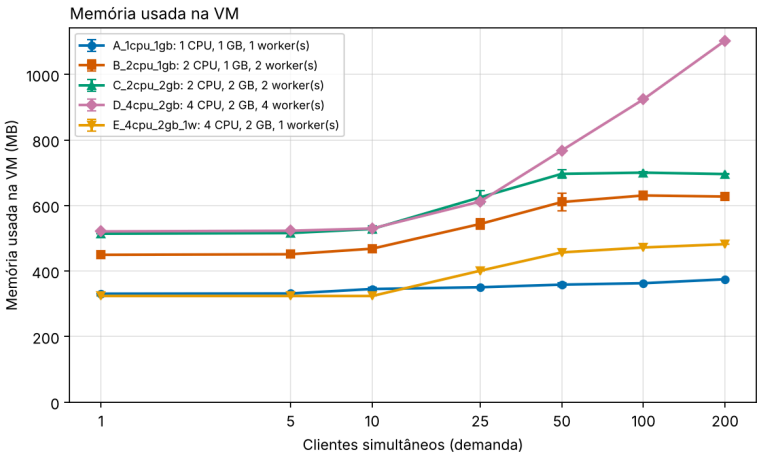
Clientes simultâneos	Vazão (req/s)	Latência média (ms)	p50 (ms)	p95 (ms)	p99 (ms)	CPU (%)	Memória usada (MB)
1	27.8	36.2	34	50	79	96.8	331
5	30.4	164.8	158	254	293	100.0	332
10	31.6	316.2	316	425	496	100.0	345
25	32.1	779.0	775	926	1004	100.0	351
50	31.8	1570.4	1590	1763	1934	100.0	359
100	31.1	3211.4	3232	3511	3609	100.0	363
200	30.0	6672.7	6442	6696	8554	100.0	375

O ponto de saturação do cenário A já aparece com 1 a 5 clientes: a vazão passa de 27,8 req/s com 1 cliente para o pico de 32,1 req/s com 25 clientes (+15%), enquanto a latência média cresce de 36 ms para 779 ms (21 vezes) no mesmo intervalo.









Comparação entre cenários

Vazão (req/s) por número de clientes simultâneos

Clientes	A (1 CPU, 1 GB, 1 worker)	B (2 CPU, 1 GB, 2 workers)	C (2 CPU, 2 GB, 2 workers)	D (4 CPU, 2 GB, 4 workers)	E (4 CPU, 2 GB, 1 worker)
1	27.8	31.7	30.8	33.1	33.8
5	30.4	40.5	40.4	44.2	27.4
10	31.6	38.8	37.3	47.1	27.6
25	32.1	37.8	29.7	44.9	27.7
50	31.8	36.8	36.3	43.6	27.3
100	31.1	36.7	35.5	41.9	26.8
200	30.0	35.9	33.9	41.6	24.7

Resumo por cenário

Cenário	Vazão máxima (req/s)	Clientes no pico	p99 com 200 clientes (ms)	CPU média da VM com 200 clientes (%)	Memória usada com 200 clientes (MB)
A	32.1	25	8554	100.0	375
B	40.5	5	7645	76.3	627
C	40.4	5	7262	72.4	696
D	47.1	10	7174	78.9	1103
E	33.8	1	10269	24.2	482

Observações sobre os dados:

- No cenário C, a repetição 3 com 25 clientes teve só 23,1 req/s (contra 32 a 34 nas outras), enquanto a interface gráfica do VirtualBox estava aberta no hospedeiro. A média de 29,7 req/s nesse nível está puxada para baixo por esse ponto, que foi mantido nos dados.
- No cenário B, o log do kernel do hospedeiro registrou um aviso de "possible SYN flooding" na porta 8080 durante os testes com 200 clientes, indicando que a fila de conexões do redirecionamento de portas encheu; mesmo assim nenhuma requisição falhou.
- Os cenários D e E têm uma única repetição por nível, então não há estimativa de variabilidade para eles. No cenário E, a CPU total fica em ~24% porque a VM tem 4 CPUs e apenas ~1 delas trabalha.

Interpretação e conclusões

A análise parte do cenário A (linha de base) e depois compara os demais. Diferenças de poucos pontos percentuais, principalmente em D e E (1 repetição), não são conclusivas.

Cenário A: o que a saturação de 1 CPU mostra

- **A CPU é o recurso limitante.** Com 1 cliente a CPU já está em 96,8% e, a partir de 5 clientes, em 100%. Como a aplicação é CPU-bound e o cliente do teste não tem tempo de pensar entre requisições, um único cliente quase satura a VM.
- **A vazão para de crescer cedo e não volta a subir.** Ela fica entre 30 e 32 req/s de 5 a 200 clientes, o que corresponde a ~31 ms de CPU por requisição, e cai cerca de 6% (32,1 para 30,0 req/s) entre 25 e 200 clientes, provavelmente pelo custo de alternar entre muitas threads e conexões.
- **Acima da saturação, a demanda extra vira fila.** A latência média é aproximadamente clientes ÷ vazão (200 ÷ 30 req/s ≈ 6,7 s), ou seja, cresce

linearmente com a demanda. O p99 passa de 79 ms (1 cliente) para 1 s (25 clientes) e 8,6 s (200 clientes).

- **Degradação de desempenho.** Com este provisionamento, a latência deixa de ser aceitável logo após 1 a 5 clientes simultâneos (p99 de 293 ms já com 5 clientes, 4 vezes o de 1 cliente); a partir de 25 clientes o p99 passa de 1 s.
- **Memória não foi gargalo.** O uso subiu de 331 MB para 375 MB (cada requisição aloca ~6 MB de crivo) e o swap ficou em zero, então 1 GB é suficiente para este cenário.
- **Variabilidade.** Com 1 cliente, a vazão variou de 25,0 a 29,6 req/s entre repetições (~17%), e no nível de 200 clientes uma repetição teve p99 de 10,7 s contra 7,3 e 7,6 s nas outras; as conclusões se apoiam nas médias e nas tendências, não em diferenças de poucos pontos percentuais.

Comparação entre os cenários

- **Mais CPUs e workers ajudam, mas menos que o proporcional.** Dobrar de 1 para 2 CPUs (A para B) elevou a vazão máxima de 32,1 para 40,5 req/s (+26%) e a vazão com 200 clientes de 30,0 para 35,9 (+20%). Com 4 CPUs e 4 workers (D), o pico foi 47,1 req/s (+47% sobre A) e 41,6 com 200 clientes (+39%).
- **A CPU da VM não chegou a saturar quando havia mais de um núcleo.** Em A a CPU fica em 100%, mas em B, C e D ela estabiliza em 72% a 79% a partir de 25 clientes, mesmo com a fila crescendo. Algo fora da CPU da VM limita a vazão. As hipóteses são que o gerador de carga (`ab`), o redirecionamento NAT do VirtualBox e a própria VM dividem os núcleos do mesmo hospedeiro, e o aviso de fila cheia na porta 8080 registrado no cenário B vai nessa direção. Os dados coletados não permitem confirmar qual dessas causas pesa mais.
- **Mais memória não ajudou uma aplicação CPU-bound.** B (1 GB) e C (2 GB), com as mesmas 2 CPUs e 2 workers, ficaram praticamente iguais: pico de 40,5 contra 40,4 req/s e p99 com 200 clientes de 7,6 s contra 7,3 s. A diferença está apenas no swap, que apareceu só em B (média de 3,5 MB com 200 clientes, e até 10,6 MB em uma execução, que teve o maior p99 de B).
- **O consumo de memória cresce com workers e concorrência.** Com 200 clientes, a memória usada foi 375 MB (A, 1 worker), 627 a 696 MB (B e C, 2 workers) e 1103 MB (D, 4 workers). Com 4 workers, 1 GB de RAM seria insuficiente e levaria a swap, então a memória passa a ser um requisito de dimensionamento quando se aumentam os workers, mesmo sem ganho direto de desempenho.
- **CPUs extras sem workers extras não servem de nada (limite do GIL).** No cenário E (4 CPUs, 1 worker), a CPU média da VM ficou em ~24%, ou seja, cerca de 1 das 4 CPUs trabalhando, e a vazão foi de ~27 req/s, abaixo da do cenário A (30 a 32 req/s) e com o pior p99 (10,3 s com 200 clientes). O fato de E ser pior que A pode vir da migração da thread entre CPUs virtuais, mas, com uma única repetição de 20 s, não dá para afirmar.
- **O ponto de degradação é parecido em todos os cenários.** A vazão atinge o pico entre 5 e 25 clientes e depois fica estável ou cai de 6% a 16% até 200 clientes, enquanto a latência cresce linearmente. O p99 passa de 1 s com cerca de 25 clientes em todos os cenários, e chega a 7 a 10 s com 200. Provisionar mais

recursos desloca pouco esse ponto: D tem p99 de 409 ms com 10 clientes, contra 496 ms em A.

Conclusões

1. **Demanda e desempenho:** acima da saturação, mais clientes não geram mais vazão, apenas mais espera. A latência média segue a Lei de Little (clientes ÷ vazão), e o p99 mostra a degradação antes da média.
2. **Provisionamento:** o gargalo depende do tipo de aplicação. Para uma aplicação CPU-bound em Python, o que importa é o número de processos (workers) alinhado ao número de CPUs; memória extra não melhora o desempenho, mas precisa cobrir o consumo dos workers.
3. **Retorno decrescente:** a vazão por CPU diminui ao escalar (32 req/s por CPU em A, 20 em B e 12 em D no pico), e na nossa configuração o ganho ficou limitado também pelo ambiente de teste (hospedeiro compartilhado entre gerador de carga e VM).
4. **Limitações:** D e E têm uma repetição de 20 s por nível; houve interferência do hospedeiro em C (25 clientes) e aviso de fila cheia em B (200 clientes); a variação entre repetições chegou a ~17% (A com 1 cliente). Para conclusões mais firmes, seria preciso gerar a carga em outra máquina e repetir D e E com 3 repetições.